MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 23 — NumPy et Pandas

Le socle de la Data Science Python — arrays et DataFrames

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

21 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 23 — NumPy et Pandas **2**
0.1 NumPy — Les tableaux multidimensionnels 2
0.2 NumPy — Opérations vectorisées 3
0.3 NumPy — Indexation et slicing avancés 4
0.4 Pandas — Series et DataFrame 5
0.5 Pandas — Filtrage et transformation 6
0.6 Pandas — Agrégations et groupby 7
0.7 Pandas — Données manquantes et fusion 8
0.8 Ce qui m'a le plus surpris dans ce chapitre 9
0.9 Résumé du Chapitre 23 10

Chapitre 23 — NumPy et Pandas

À retenir

NumPy et **Pandas** sont les deux bibliothèques fondamentales de la Data Science en Python. NumPy fournit des tableaux multidimensionnels ultra-performants pour le calcul numérique. Pandas construit par-dessus NumPy pour offrir des structures de données tabulaires — les `DataFrame` et `Series` — pensées pour l'analyse de données. Ensemble, ils permettent de manipuler des millions de lignes en quelques millisecondes.

NumPy — Les tableaux multidimensionnels

numpy — créer des arrays

```python
import numpy as np

# Creer un array depuis une liste
a = np.array([1, 2, 3, 4, 5])
print(a)            # [1 2 3 4 5]
print(type(a))      # <class 'numpy.ndarray'>
print(a.dtype)      # int64
print(a.shape)      # (5,)
print(a.ndim)       # 1

# Array 2D      matrice
m = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print(m.shape)      # (3, 3)
print(m.ndim)       # 2
print(m.size)       # 9

# Fonctions de creation
zeros    = np.zeros((3, 4))          # matrice de zeros
uns      = np.ones((2, 3))           # matrice de uns
identite = np.eye(4)                 # matrice identite
plage    = np.arange(0, 20, 2)       # [0 2 4 ... 18]
lineaire = np.linspace(0, 1, 11)     # 11 points de 0 a 1
aleatoire = np.random.rand(3, 3)     # valeurs al atoires [0,1)

print(plage)
print(lineaire)

# Changer le type
entiers  = np.array([1, 2, 3])
```

```python
flottants = entiers.astype(float)
print(flottants)    # [1. 2. 3.]
```

NumPy — Opérations vectorisées

Calculs vectorisés — sans boucles

```python
import numpy as np

a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])

# Operations element par element      SANS BOUCLE
print(a + b)        # [11 22 33 44 55]
print(a * b)        # [ 10  40  90 160 250]
print(a ** 2)       # [ 1  4  9 16 25]
print(b / a)        # [10. 10. 10. 10. 10.]

# Broadcasting      operation avec un scalaire
print(a * 3)        # [ 3  6  9 12 15]
print(a + 100)      # [101 102 103 104 105]

# Comparaison vectorisee
print(a > 3)        # [False False False  True  True]
print(a[a > 3])     # [4 5]       filtrage booleen !

# Fonctions universelles (ufunc)
print(np.sqrt(a))              # [1.  1.41 1.73 2.  2.24]
print(np.log(b))               # logs naturels
print(np.sin(np.linspace(0, np.pi, 5)))

# Statistiques
notes = np.array([15, 18, 12, 9, 17, 14, 11, 16])
print(f"Moyenne  : {np.mean(notes):.2f}")
print(f"M diane  : {np.median(notes):.2f}")
print(f" cart -type : {np.std(notes):.2f}")
print(f"Min / Max  : {np.min(notes)} / {np.max(notes)}")
print(f"Percentile 75 : {np.percentile(notes, 75)}")

# Performance vs liste Python
import time
n = 1_000_000
liste = list(range(n))
array = np.arange(n)

t0 = time.time()
resultat_liste = [x**2 for x in liste]
print(f"Liste  : {time.time()-t0:.4f}s")
```

```python
t0 = time.time()
resultat_array = array**2
print(f"NumPy  : {time.time()-t0:.4f}s")
# NumPy est ~50x plus rapide !
```

NumPy — Indexation et slicing avancés

Indexation multidimensionnelle

```python
import numpy as np

m = np.array([[1,  2,  3,  4],
              [5,  6,  7,  8],
              [9,  10, 11, 12]])

# Acces par index [ligne, colonne]
print(m[0, 0])     # 1
print(m[1, 2])     # 7
print(m[-1, -1])   # 12

# Slicing 2D [lignes, colonnes]
print(m[0, :])     # [1 2 3 4]         premiere ligne
print(m[:, 0])     # [1 5 9]           premiere colonne
print(m[1:, 2:])   # [[7 8],[11 12]]

# Filtrage booleen
print(m[m > 6])    # [ 7  8  9 10 11 12]
m[m < 5] = 0       # remplacer les valeurs < 5 par 0
print(m)

# Reshape      changer la forme sans copier
a = np.arange(12)
print(a.reshape(3, 4))    # (3,4)
print(a.reshape(2, 6))    # (2,6)
print(a.reshape(2, 2, 3)) # (2,2,3)      3D !

# Transposer une matrice
mat = np.array([[1, 2, 3],
                [4, 5, 6]])
print(mat.T)    # Transposee (3,2)

# Aplatir en 1D
print(mat.flatten())    # [1 2 3 4 5 6]
print(mat.ravel())      # vue aplatie (pas de copie)

# Concatener des arrays
a = np.array([[1, 2], [3, 4]])
```

```
b = np.array([[5, 6], [7, 8]])
print(np.vstack([a, b]))    # empiler verticalement
print(np.hstack([a, b]))    # empiler horizontalement
```

Pandas — Series et DataFrame

pandas — Series

```
import pandas as pd
import numpy as np

# Series      tableau 1D avec index
notes = pd.Series([15, 18, 12, 9, 17],
                  index=["Alice", "Bob", "Charlie",
                         "Diana", "Eve"])
print(notes)
print(notes.dtype)        # int64
print(notes.index)        # Index(['Alice', ...])

# Acces
print(notes["Alice"])     # 15
print(notes[0])           # 15  (index entier)
print(notes[notes > 14]) # Alice:15, Bob:18, Eve:17

# Operations
print(notes.mean())       # 14.2
print(notes.max())        # 18
print(notes + 2)          # toutes les notes + 2
print(notes * 5)          # sur 100

# Valeurs manquantes
avec_nan = pd.Series([15, None, 12, None, 17])
print(avec_nan.isna())         # True la ou c'est NaN
print(avec_nan.fillna(0))      # remplacer NaN par 0
print(avec_nan.dropna())       # supprimer les NaN
print(avec_nan.interpolate()) # interpoler les NaN
```

pandas — DataFrame

```
import pandas as pd

# DataFrame      tableau 2D avec index et colonnes
data = {
    "prenom":  ["Alice", "Bob", "Charlie", "Diana", "Eve"],
    "age":     [25, 30, 22, 28, 35],
    "ville":   ["Paris", "Lyon", "Paris", "Marseille", "Lyon"],
    "note":    [15, 18, 12, 9, 17],
```

```
    "salaire": [45000, 52000, 38000, 41000, 61000]
}

df = pd.DataFrame(data)
print(df)
print(df.shape)     # (5, 5)
print(df.dtypes)
print(df.info())    # resume complet

# Acceder aux colonnes
print(df["prenom"])            # Series
print(df[["prenom", "note"]]) # DataFrame

# Acceder aux lignes
print(df.loc[0])               # par label/index
print(df.iloc[0])              # par position entiere
print(df.loc[0, "prenom"])     # Alice

# Slicing
print(df.iloc[1:4])            # lignes 1 a 3
print(df.loc[:, "age":"note"]) # colonnes age a note
```

Pandas — Filtrage et transformation

Filtrer et transformer un DataFrame

```
import pandas as pd

df = pd.DataFrame({
    "prenom":  ["Alice", "Bob", "Charlie", "Diana", "Eve"],
    "ville":   ["Paris", "Lyon", "Paris", "Marseille", "Lyon"],
    "note":    [15, 18, 12, 9, 17],
    "salaire": [45000, 52000, 38000, 41000, 61000]
})

# Filtrage booleen
admis = df[df["note"] >= 10]
print(admis)

parisiens = df[df["ville"] == "Paris"]
print(parisiens)

# Plusieurs conditions
elite = df[(df["note"] >= 15) & (df["salaire"] > 40000)]
print(elite)

# query()      syntaxe plus lisible
lyonnais_admis = df.query("ville == 'Lyon' and note >= 10")
```

```python
print(lyonnais_admis)

# Ajouter une colonne calculee
df["note_sur_100"] = df["note"] * 5
df["mention"] = df["note"].apply(
    lambda n: "TB" if n >= 16 else
              "B"  if n >= 14 else
              "AB" if n >= 12 else
              "P"  if n >= 10 else "F"
)
print(df[["prenom", "note", "mention", "note_sur_100"]])

# Trier
print(df.sort_values("note", ascending=False))
print(df.sort_values(["ville", "note"]))

# Renommer des colonnes
df = df.rename(columns={"prenom": "nom", "note": "score"})

# Supprimer des colonnes
df = df.drop(columns=["note_sur_100"])
```

Pandas — Agrégations et groupby

groupby — analyser par catégories

```python
import pandas as pd

df = pd.DataFrame({
    "prenom":  ["Alice","Bob","Charlie","Diana","Eve","Frank"],
    "ville":   ["Paris","Lyon","Paris","Marseille","Lyon","
       Paris"],
    "dept":    ["Info","Math","Info","Math","Info","Math"],
    "note":    [15, 18, 12, 9, 17, 14],
    "salaire": [45000, 52000, 38000, 41000, 61000, 48000]
})

# Statistiques descriptives globales
print(df.describe())
print(df["note"].value_counts())

# groupby      regrouper par ville
par_ville = df.groupby("ville")["note"]
print(par_ville.mean())     # moyenne par ville
print(par_ville.max())      # max par ville
print(par_ville.count())    # effectif par ville

# Plusieurs agregations
```

```python
resume = df.groupby("ville").agg({
    "note":    ["mean", "min", "max"],
    "salaire": ["mean", "sum"]
})
print(resume)

# groupby multi-colonnes
par_dept_ville = df.groupby(["dept", "ville"])["note"].mean()
print(par_dept_ville)

# pivot_table     tableau croise
pivot = df.pivot_table(
    values="note",
    index="ville",
    columns="dept",
    aggfunc="mean",
    fill_value=0
)
print(pivot)

# apply()     fonction personnalisee par groupe
def stats_groupe(groupe):
    return pd.Series({
        "moyenne": groupe["note"].mean(),
        "admis":   (groupe["note"] >= 10).sum(),
        "total":   len(groupe)
    })

print(df.groupby("ville").apply(stats_groupe))
```

Pandas — Données manquantes et fusion

Gérer les NaN et fusionner des DataFrames

```python
import pandas as pd
import numpy as np

# Donnees manquantes
df = pd.DataFrame({
    "prenom": ["Alice", "Bob", "Charlie", "Diana"],
    "note":   [15, np.nan, 12, np.nan],
    "ville":  ["Paris", None, "Lyon", "Paris"]
})

print(df.isna())            # masque booleen
print(df.isna().sum())      # nb de NaN par colonne
print(df.isna().sum().sum()) # total NaN
```

```python
# Traiter les NaN
df["note"].fillna(df["note"].mean(), inplace=True)
df["ville"].fillna("Inconnue", inplace=True)
print(df)

# Fusionner deux DataFrames
etudiants = pd.DataFrame({
    "id":     [1, 2, 3, 4],
    "prenom": ["Alice", "Bob", "Charlie", "Diana"]
})

notes = pd.DataFrame({
    "id":     [1, 2, 3, 5],
    "note":   [15, 18, 12, 14]
})

# merge()     comme un JOIN SQL
inner = pd.merge(etudiants, notes, on="id", how="inner")
left  = pd.merge(etudiants, notes, on="id", how="left")
outer = pd.merge(etudiants, notes, on="id", how="outer")

print("Inner join :\n", inner)
print("\nLeft join  :\n", left)
print("\nOuter join :\n", outer)

# concat()     empiler des DataFrames
df1 = pd.DataFrame({"prenom": ["Alice"], "note": [15]})
df2 = pd.DataFrame({"prenom": ["Bob"],   "note": [18]})
df3 = pd.concat([df1, df2], ignore_index=True)
print(df3)
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. La vectorisation NumPy — 50x plus rapide sans boucle. Écrire `array ** 2` au lieu de `[x**2 for x in liste]` n'est pas juste plus court. C'est une opération qui s'exécute en C compilé, sur des blocs de mémoire contiguë, avec des instructions SIMD. La différence de performance m'a stupéfié : un million d'éléments en 2ms avec NumPy vs 100ms avec Python pur. La vectorisation n'est pas une optimisation — c'est un paradigme différent.

2. `loc` vs `iloc` — une distinction capitale. `df.loc[0]` accède par **label** d'index. `df.iloc[0]` accède par **position entière**. Quand l'index est numérique par défaut, les deux semblent identiques. Mais après un `sort_values()` ou un `reset_index()`, l'index peut ne plus correspondre aux positions. J'ai eu des bugs silencieux en confondant les deux — et une mauvaise compréhension qui a duré longtemps.

3. `groupby` est une révolution pour l'analyse statistique. En tant qu'étudiant en statistique, voir que `df.groupby("ville")["note"].mean()` calcule instantanément la moyenne par ville sur un million de lignes m'a montré la puissance de Pandas pour la statistique appliquée. Ce que je faisais avec des boucles imbriquées en dix lignes, Pandas le fait en une. C'est la promesse de la Data Science Python concrètement réalisée.

Résumé du Chapitre 23

Ce que j'ai appris dans ce chapitre :

- **NumPy** : `ndarray` — tableaux N-D vectorisés, performants.
- Opérations vectorisées — calculs sans boucle, 50x plus rapides.
- Filtrage booléen : `array[array > 5]` — sélection expressive.
- **Pandas Series** — tableau 1D avec index nommé.
- **Pandas DataFrame** — tableau 2D : `loc`, `iloc`, filtrage.
- `apply()` et `query()` — transformer et filtrer élégamment.
- `groupby()` et `agg()` — agrégations statistiques par groupe.
- `merge()` et `concat()` — fusionner des DataFrames comme en SQL.
- Données manquantes : `isna()`, `fillna()`, `dropna()`.

"NumPy donne la vitesse. Pandas donne l'expressivité. Ensemble, ils transforment Python en outil d'analyse de données de classe mondiale."